CSE 341 · GEBZE TECHNICAL UNIVERSITY · SPRING 2026

Design & Implement Your Own Language

A complete guide to the Mini Programming Language project — covering everything from domain selection to the final written exam.

PART 1 DUE	EXAM 1	PART 2 DUE	EXAM 2
08 May 2026	14 May 2026	22 May 2026	04 Jun 2026

01 Project Overview

2
SUBMISSION PARTS

2
WRITTEN EXAMS

6

DELIVERABLES

1–2

TEAM SIZE

🎯 What Is This Project?

You will design your own **Domain-Specific Language (DSL)** and build a full interpreter for it. The project covers Chapters 1–7 of Sebesta's *Concepts of Programming Languages* (12th edition).

The project is split into two parts:

- **Part 1:** Language design + Lexer + Parser
- **Part 2:** Type checker + Interpreter + Full execution

⚠️ The AI-Resistant Mechanism

After each submission, you take a **90-minute in-class written exam** with questions *tailored specifically to your own submission.*

Example question: *"In your D1 you said your language has static scoping — show on this code excerpt how the lookup of variable x resolves."*

If you can't answer questions about your own work, your grade drops — regardless of how polished the submission looks.

Core Goal: The primary deliverable is your *design decisions* and your ability to *defend them* using the vocabulary of Sebesta's textbook. No AI can complete this end-to-end because it requires *your understanding*.

What Your Language Must Have (Minimum Requirements)

Type System

- At least **3 primitive types**
- At least **1 structured type** (array, record, or user-defined)

Control Flow

- At least **1 control structure** beyond sequencing
- Examples: if, while, for, pattern-match

Functions

- **User-defined functions or procedures** with parameters

Domain-Specific Construct

- At least **one DSL construct** that justifies why this isn't a generic scripting language

Expression Language

- Nontrivial operators raising questions about **precedence, associativity, mixed-mode arithmetic, coercion**

Example Domain Ideas (from the Handout)

TurtleLang — turtle graphics
ChessDSL — chess puzzles
QueryLite — CSV queries
MusicScript — melodies
CircuitLang — digital circuits
RecipeLang — cooking/ingredients
PlotDSL — 2D data plots
Your own original domain ✨

02 Complete Timeline

MON 04 MAY 2026

⚡ Pair Declaration Deadline

Submit the pair declaration form. Students who miss this are treated as solo. Each partner writes their own AI Usage Journal — it is never shared.

FRI 08 MAY 2026, 23:59

📦 Part 1 Submission

D1 (design spec, Ch.1–5 sections), D2 (lexer + parser), D3 (test report), D4 (AI journal ≥4 entries + E1), D5 (retrospective ½ page), D6 (pairs only).

THU 14 MAY 2026, 08:30 — 90 min

📝 Exam 1 (In-Class)

Covers your Part 1 submission + Sebesta Ch. 1, 3, 4, 5. Questions tailored to your own design. Bring up to 3 sheets of handwritten A4 notes (one side each). No printed material.

FRI 22 MAY 2026, 23:59

📦 Part 2 Submission

Complete spec (Ch.1–7), full interpreter, updated test report (executed programs + type error), AI journal (≥6 more entries + E2, E3), full retrospective, D6 update.

THU 04 JUN 2026 — Class Hours (Makeup Week)

📝 Exam 2 (In-Class)

Covers your full Part 2 submission + Sebesta Ch. 6, 7. May reference Part 1 material. Questions tailored to your own submission. Same format as Exam 1.

⏱ Late Policy

−10% per calendar day late, up to 3 days. Zero credit after 3 days.

Applied *per part separately*. Extensions require documented medical/personal justification and must be requested *before* the relevant deadline.

📋 Exam Format

- Written, on paper — same lined-and-bordered style as the midterm
- Mix of short-answer, draw, annotate questions
- **What you may bring:** 3 sheets of A4 handwritten notes, one side each. Your name + student ID at the top of each sheet.
- **What you may NOT bring:** Printed material, photocopied notes, laptops, phones, books

- All notes are collected at the end of the exam

🔄 Part 2 = Revised Part 1

Part 2 is a **re-submission of the entire project**. You may freely revise everything from Part 1. The two parts are **weighted equally** in the final grade.

Your Part 2 spec must be *consistent* with your Part 2 implementation. You can revise the spec — but not retroactively after submission.

03 All Deliverables

CODE	NAME	PART 1 – DUE 8 MAY	PART 2 – DUE 22 MAY	WHO SUBMITS
D1	**Design Specification**	Language overview, lexical structure, EBNF syntax, names/ bindings/ scope/ lifetime (Ch. 1–5). P1	Complete final version + type system, expressions, semantics for 2 constructs, full rationale (Ch.1–7). P2	Both partners individually
D2	**Implementation (Source Code)**	Working lexer + parser.	Adds type checker + interpreter.	One partner (pairs)

CODE	NAME	PART 1 – DUE 8 MAY	PART 2 – DUE 22 MAY	WHO SUBMITS
		Accepts valid programs → printable AST. Rejects malformed programs with error + line number. README with build/run instructions. P1	End-to-end execution of 3 example programs. Informative runtime errors. P2	
D3	**Example Programs & Test Report**	3 non-trivial programs that parse successfully + 5 malformed programs showing error messages with line numbers. PDF, 2–3 pages. P1	Same 3 programs now executed with actual output + brief discussion. At least 1 type-error program. PDF, 3–5 pages. P2	One partner (pairs)
D4	**AI Usage Journal *(individual!)***	Min. **4 substantive entries**, dated up to 8 May. Must include Experiment E1. P1	Min. **6 additional entries**, dated 9–22 May. Must include Experiments E2 and E3. P2	Each student individually

CODE	NAME	PART 1 – DUE 8 MAY	PART 2 – DUE 22 MAY	WHO SUBMITS
D5	**Retrospective & Self-Assessment**	½ page: what is working, what is hard, what you plan to fix. P1	1 full page: retrospective (½) + honest self-assessment against rubric grading yourself A/B/C/D with justification (½). P2	Each student individually
D6	**Contribution Report *(pairs only)***	For each D1 section and D2 component: who wrote it and who reviewed it. State workload is evenly split. ½ page. P1	Updated report covering full project. Each partner ≥40% of total work. P2	One partner (pairs)

File Naming Convention: Every file starts with your student number (solo) or both IDs joined by a hyphen (pairs), then an underscore, then the deliverable code.
`20210045_D1_P1.pdf` / `20210045-20210078_D2_P2.zip`
For Part 2: replace `_P1` with `_P2`. Submit via **Microsoft Teams → CSE 341 → Assignments tab**. No email submissions.

Design Specification (D1) — Section-by-Section Breakdown

§4.1 Language Overview P1

- Name, domain, sample program
- Justify using Sebesta's **language evaluation criteria** (Ch.1): readability, writability, reliability, cost
- Which criteria does your DSL prioritize? Which does it knowingly sacrifice?

§4.2 Lexical Structure P1

- List all token categories: identifiers, literals, keywords, operators, separators
- Give regex/pattern for each

§4.3 Syntax P1

- Complete grammar in **EBNF** (Sebesta §3.1–3.2)
- Must be unambiguous for the subset you implement
- Annotate ambiguity resolutions: operator precedence, dangling else

§4.4 Semantics P2

- Operational or denotational semantics (Sebesta §3.5)
- For at least **2 non-trivial constructs** (e.g., while loop, DSL construct)
- ⛔ Axiomatic semantics is NOT accepted

§4.5 Type System P2

- Primitive types + value sets
- Strongly typed? (§6.12) — justify
- Implicit coercion? (§7.4) — state rules
- Name vs. structural equivalence (§6.14) — why?
- Structured type design decisions (§6.5–6.7)

§4.6 Names, Binding, Scope, Lifetime P1

- Legal identifier rules
- Compile-time vs. run-time bindings
- **Static or dynamic scoping?** Why? What breaks if you switch?
- Lifetime of locals: stack-dynamic, static, or explicit-heap-dynamic?

§4.7 Expressions & Assignment P2

- Operator precedence and associativity table
- Short-circuit Boolean evaluation?
- Order of operand evaluation: defined or unspecified?
- Assignment: statement or expression?

§4.8 Design Rationale P2

- For each of **5 numbered decisions** from §4.5–4.7: one paragraph justification + one named trade-off

- This is where AI-generated boilerplate fails most visibly — write in your own voice

04 Implementation Guide

🔧 Part 1 Implementation (D2-P1)

- **README** with build + run instructions and the exact command line used
- **Lexer:** tokenizes all valid programs
- **Parser:** accepts valid programs → produces printable AST (`--dump-ast` flag)
- **Error handling:** rejects malformed programs with *useful error message including line number*
- Must handle all 3 example programs from D3

🚀 Part 2 Implementation (D2-P2)

- **Type Checker:** consistent with D1 type system (strong/weak typing, coercion rules, type equivalence)
- **Interpreter:** executes programs respecting declared scoping, precedence, associativity, parameter-passing mode
- **Runtime errors:** division by zero, index out of range (if applicable), at least 1 domain-specific error
- **End-to-end:** all 3 example programs execute with expected outputs

💻 Allowed Implementation Languages

Python | JavaScript / TypeScript | C | C++ | Java | C# | Go | Rust | OCaml | Haskell

ANTLR / yacc / bison / pegjs (parser generators — allowed if you can explain them in exam)

Critical: Consistency Between D1 and D2

Your implementation must match the language you described in D1. If D1 says "static scoping" but your interpreter resolves variables dynamically, *both D1 and D2 lose points*. You are allowed to revise D1 — but the final submitted spec must match the final submitted implementation.

Recommended Architecture

Lexer

Tokenize source → stream of tokens. Handle identifiers, literals, keywords, operators, separators. Emit errors with line numbers.

Parser

Tokens → AST. Implement your EBNF grammar. Handle precedence, associativity. `--dump-ast` flag required.

Type Checker

Walk AST → verify types per your declared rules. Emit type errors before execution. Consistent with §4.5 of D1.

Interpreter

Walk AST → execute. Respect scoping, parameters, precedence. Produce informative runtime errors. Output matches D3.

05 Sebesta Book — Chapter Map

The project covers Chapters 1–7 of Sebesta's *Concepts of Programming Languages (12th ed.)*. Chapters 1–5 are covered in Part 1 (Exam 1); Chapters 6–7 are added in Part 2 (Exam 2).

CHAPTER 1

Preliminaries

Exam 1

Language evaluation criteria (readability, writability, reliability, cost), language categories, design trade-offs, implementation methods. Used in D1 §4.1.

CHAPTER 2

Evolution of Major Languages

Exam 1

Historical context: Fortran, Lisp, COBOL, ALGOL, C, Smalltalk, Java, C#. Background for design decisions.

CHAPTER 3

Describing Syntax and Semantics

Exam 1

EBNF (§3.1–3.2) for D1 §4.3. Attribute grammars. Dynamic semantics (§3.5) for D1 §4.4 — operational or denotational only.

CHAPTER 4

Lexical and Syntax Analysis

Exam 1

Lexical analysis, parsing problem, recursive-descent parsing, bottom-up parsing. Directly implements D2 lexer + parser.

CHAPTER 5

Names, Bindings, and Scopes

Exam 1

Variables, binding, scope, scope and lifetime, referencing environments, named constants. Used in D1 §4.6 — static vs. dynamic scoping, stack-dynamic lifetime.

CHAPTER 6

Data Types

Exam 2

Primitive types, strings, arrays (§6.5), records (§6.6–6.7), tuples, unions. Strong typing (§6.12), type equivalence (§6.14 — name vs. structural). Used in D1 §4.5.

CHAPTER 7

Expressions and Assignment

Exam 2

Arithmetic expressions, overloaded operators, type conversions/coercion (§7.4), relational + Boolean expressions, short-circuit evaluation, assignment statements. Used in D1 §4.5 and §4.7.

Key Sebesta Concepts You MUST Know Per Deliverable Section

For D1 §4.6 — Scope & Binding

- **Static (lexical) scoping** vs. dynamic scoping — how variable lookup works in each
- **Stack-dynamic** vs. static vs. heap-dynamic lifetime
- **Compile-time** vs. run-time binding
- Be ready to trace variable resolution on code examples from your own language

For D1 §4.5 — Type System

- **Strong typing (§6.12)** — definition and what makes a language strongly typed
- **Name equivalence vs. structural equivalence (§6.14)** — how they differ with record types
- **Implicit coercion (§7.4)** — widening vs. narrowing, type coercion rules
- Array design: static vs. dynamic, index range, subscript checking (§6.5)

06 AI Usage Journal

AI use is MANDATORY and GRADED. You are required to use AI assistants (Claude, ChatGPT, Copilot, Gemini, etc.) on this project. The

> point is to train you to use AI like an engineer — critically, with judgment, with full understanding of the output.

≥4

ENTRIES BY 8 MAY

≥6

MORE ENTRIES BY 22 MAY

3

MANDATORY EXPERIMENTS

Three Mandatory Experiments

EXPERIMENT E1 — Required in Part 1 Journal

Ask AI to generate an EBNF grammar for your language

Give the AI a one-paragraph description of your language. Ask it to generate the complete EBNF grammar.

📊 Report: Compare its grammar with yours. What did it get wrong or suboptimal? Does it handle operator precedence correctly?

EXPERIMENT E2 — Required in Part 2 Journal

Ask AI to explain name vs. structural equivalence for your record type

Ask the AI to explain the difference between name equivalence and structural equivalence specifically for your language's record/structured type.

📊 Report: Does its explanation match Sebesta §6.14? Does the example it gives actually run under the equivalence rule it described?

EXPERIMENT E3 — Required in Part 2 Journal

Ask AI to implement your type checker (or a non-trivial portion)

Ask the AI to generate the type checker code — or a significant portion of it.

📊 Report: Run its code on ≥3 test inputs where you know the right answer. Report any bugs (subtle or blatant) and how you fixed them.

Journal Entry Template

Entry #	e.g., "7 of 14"
Date	YYYY-MM-DD — **MUST be a real date during project work**
Phase	Design / Lexer / Parser / Type checker / Interpreter / Testing / Writing
AI Tool	Claude Sonnet / ChatGPT / Copilot / Gemini / other (+ version if known)
Goal	One sentence — what you were trying to achieve
Prompt	Verbatim. If you iterated, show the final prompt + one-line summary of what changed.
Response	Verbatim code/text or tight summary with direct quotes of important lines

Accepted	What you kept, and *why*
Rejected / Modified	What you threw away or changed, and *why*. Be specific: bug, wrong abstraction, hallucinated API…
Errors You Caught	Mistakes the AI made. How did you detect them? What would have happened if you hadn't?
Reflection	2–3 sentences. What did you learn about the problem — or about using AI?

Backfilled journals receive ZERO. Every entry must carry a real date. A journal whose entries are all dated in the final 48 hours is not credible.

Journal must include ≥3 cases where AI was WRONG, unclear, or suboptimal. Entries that just say "AI gave correct answer, I used it" get minimum credit.

07 Grading Rubric

Both parts are graded on all 6 dimensions below. Written artifacts → dimensions 1–5. In-class written exam → dimension 6. **The two parts are weighted equally** in the final project grade.

- **1. Correctness**

A – Excellent	B – Good	C – Adequate	D/F – Weak
All stated behaviors work; no contradictions between spec and implementation.	Minor bugs; ≤2 contradictions that are clearly acknowledged.	Core features work; edge cases fail; some inconsistencies with spec.	Spec and implementation diverge significantly; major features missing.

2. Design Justification

A – Excellent	B – Good	C – Adequate	D/F – Weak
Every major decision justified with Sebesta terminology and a named trade-off.	Most decisions justified; terminology mostly correct.	Generic justifications; terminology used loosely.	Decisions stated without justification, or justifications are boilerplate.

3. Use of Sebesta Framework

A – Excellent	B – Good	C – Adequate	D/F – Weak
Accurate use of grammar formalism, semantics style, type-system and binding terminology.	Accurate terminology with 1–2 misapplications.	Terminology mostly present but shallowly applied.	Terminology misused or absent.

4. Originality

A – Excellent	B – Good	C – Adequate	D/F – Weak
Clearly distinct DSL with a genuine domain-	Distinct DSL; domain-specific construct is present but thin.	DSL is a lightly themed reskin of a generic language.	Indistinguishable from a generic textbook toy language.

specific construct.			

5. Code Quality

A – Excellent	B – Good	C – Adequate	D/F – Weak
Clean architecture; lexer, parser, type checker, evaluator clearly separated.	Readable code with working separation of concerns.	Code works but is tangled; separation unclear.	Code is opaque, undocumented, or doesn't match spec.

6. Exam Performance

A – Excellent	B – Good	C – Adequate	D/F – Weak
Confidently answers project-specific + general PL questions; can explain alternatives not taken.	Answers most questions correctly; recovers from follow-ups with light prompting.	Answers surface questions; cannot discuss design alternatives or apply to own code.	Cannot answer fundamental questions about own submission or corresponding chapters.

Exam ↔ Artifact Interaction: If exam answers show you don't understand a section of your own submission at a basic level, the instructor may reduce the corresponding artifact grade. Conversely, strong exam answers can *recover* points on written work with unclear justification.

08 Rules, Prohibitions & Permissions

✅ PERMITTED / MANDATORY

AI Tool Use — REQUIRED

You MUST use AI assistants (Claude, ChatGPT, Copilot, Gemini, etc.) and document every substantive interaction in your AI Usage Journal.

AI-Assisted Code

AI-generated code is fine — but you must be able to explain every line you submit in the exam.

Parser Generators

ANTLR, yacc/bison, pegjs are allowed as long as you can explain them during the exam.

Free Choice of Language

Python, JavaScript/TypeScript, C, C++, Java, C#, Go, Rust, OCaml, Haskell are all allowed.

3 Handwritten A4 Sheets in Exam

One side each. Must have your name + student ID. Collected at end of exam.

Revise D1 in Part 2

All Part 1 sections of D1 may be freely revised in Part 2 — as long as the final spec matches the final implementation.

⛔ PROHIBITED

Pasting AI text without marking it

You may NOT paste AI-generated text directly into D1, D3, or journal reflections without marking it as a quotation AND discussing it. Rationale sections must be in your own voice.

Copying from other students

Copying another student's (or pair's) design, code, or rationale is plagiarism. Suspiciously similar submissions will both be investigated.

Backfilling the AI Journal

Undated or last-minute journals (all entries in final 48 hours) receive ZERO. Dates must be real and spread across the working period.

Axiomatic Semantics in D1 §4.4

Only operational or denotational semantics accepted for this project.

Printed / photocopied notes in exam

Only handwritten notes allowed. No laptops, phones, books, or any printed material during the exam.

Submitting by email

All submissions must go through Microsoft Teams → CSE 341 → Assignments tab. Email submissions are not accepted.

Pair Work Rules

Individual per-partner (always)

- D1 Design Specification — each partner submits their own
- D4 AI Usage Journal — always individual, even in pairs
- D5 Retrospective — each partner writes their own
- Written Exam — each partner answers questions about *their own* components

One partner submits (pairs)

- D2 Implementation zip — one upload
- D3 Test Report — one upload

- D6 Contribution Report — one upload, signed by both
- Each partner must be ≥40% of total work. Lopsided splits are detected through exams and graded down individually.

09 Submission Checklists

📦 Before Submitting Part 1 (8 May)

- ☐ Pair declaration form submitted (if pair) — by 4 May
- ☐ D1 PDF includes: language overview, lexical structure, EBNF grammar, names/binding/scope/lifetime
- ☐ D1 cites Sebesta terminology correctly (readability, writability, static scoping, etc.)
- ☐ D2 zip contains README with build + run instructions
- ☐ D2 lexer tokenizes all valid programs
- ☐ D2 parser produces printable AST with `--dump-ast` or equivalent
- ☐ D2 rejects malformed input with error message + line number
- ☐ D3: 3 non-trivial programs parse successfully
- ☐ D3: 5 malformed programs with error messages + line numbers
- ☐ D4: ≥4 journal entries, real dates spread out, includes Experiment E1

- ☐ D4: ≥3 cases where AI was wrong/suboptimal documented
- ☐ D5: ½ page retrospective
- ☐ D6 (pairs only): contribution report, both sections covered, workload stated
- ☐ All filenames start with student ID(s) + deliverable code + _P1
- ☐ All PDFs open and not password-protected
- ☐ Uploaded to Teams "Project Part 1" assignment — NOT by email

🚀 Before Submitting Part 2 (22 May)

- ☐ D1 updated: adds semantics (2 constructs), type system, expressions section, design rationale
- ☐ D1 spec is consistent with D2 implementation (scoping, typing, precedence)
- ☐ D2 type checker consistent with D1 §4.5
- ☐ D2 interpreter respects declared scoping rule and precedence
- ☐ D2 runtime errors: division by zero, index OOB, ≥1 domain-specific error
- ☐ D2 executes all 3 example programs end-to-end with expected output
- ☐ D3 updated: 3 programs with actual executed output + discussion
- ☐ D3: ≥1 program triggering a type error before execution

- ☐ D4 continuation: ≥6 more entries dated 9–22 May, includes E2 and E3
- ☐ D5: full 1-page retrospective + honest rubric self-assessment (A/B/C/D per dimension)
- ☐ D6 (pairs) updated: covers full project, both partners ≥40%
- ☐ All filenames use _P2
- ☐ Uploaded to Teams "Project Part 2" assignment
- ☐ All PDFs open and not password-protected
- ☐ D1 design rationale written in your own voice (not AI-generated boilerplate)

📝 Exam Preparation Checklist

For Exam 1 (14 May) — know your Part 1

- ☐ Can explain your scoping rule with a concrete code example from your language
- ☐ Can trace how a variable lookup resolves step-by-step
- ☐ Can explain every rule in your EBNF grammar
- ☐ Can explain how your parser handles operator precedence
- ☐ Know Sebesta Ch. 1, 3, 4, 5 terminology cold
- ☐ Prepared 3 handwritten A4 note sheets with name + ID

- ☐ Know what "stack-dynamic lifetime" means in your own language

For Exam 2 (04 Jun) — know your Part 2

- ☐ Can explain your type equivalence rule (name vs. structural) with an example
- ☐ Can show how your type checker handles a specific expression
- ☐ Can explain coercion rules with a concrete example from your language
- ☐ Can describe how your interpreter evaluates short-circuit Boolean expressions
- ☐ Know Sebesta Ch. 6, 7 terminology cold
- ☐ Can trace execution of your interpreter on a code snippet
- ☐ Prepared 3 handwritten A4 note sheets for Exam 2